AI-Powered Image Quality & Defect Detection

Internship Applicant Technical Assessment

Assessment duration	48 hours
Primary focus	Computer Vision, ML / Deep Learning, Backend, Frontend & Deployment
External AI services	Not permitted
API keys	Not required
Expected outcome	A working, deployable full-stack AI application

1. Problem Statement

Build a full-stack application that accepts an image and automatically evaluates its visual quality. The system should identify common image-quality problems and determine whether the image is acceptable, degraded, or potentially defective. The solution must demonstrate meaningful use of computer vision and machine learning or deep learning without relying on external AI or vision APIs.

2. Required Detection Capabilities

- Blur / insufficient sharpness
- Underexposure
- Overexposure
- Image noise
- Image corruption or severe degradation
- Potential visual defect

Additional quality issues may be identified if technically justified.

3. AI / Computer Vision Requirements

The application must include an AI-based decision component. A traditional computer-vision-only solution is not sufficient for full credit. Applicants may choose an appropriate approach.

- Classical machine learning using engineered image features.
- A lightweight deep-learning model or transfer-learning approach using PyTorch, TensorFlow, or another suitable framework.
- A hybrid approach combining image-quality features with a learned model.
- An anomaly-detection approach, provided the formulation and evaluation are clearly explained.

Applicants should explain model selection, data preparation, training or model acquisition, and evaluation.

4. Image Analysis

The system should derive meaningful information from the input image. Applicants should demonstrate understanding of characteristics such as sharpness, brightness/exposure, contrast, noise, texture, saturation, or other features relevant to their selected method. The exact feature set and modelling strategy are intentionally open.

5. Backend Requirements

- Provide a REST API for image upload and analysis.
- Validate uploaded files and handle invalid or unreadable images gracefully.
- Return a structured analysis result in JSON.
- Persist analysis results in SQLite, PostgreSQL, or another suitable database.
- Provide an endpoint to retrieve previous analysis results.
- Include appropriate error handling and HTTP status codes.

6. Frontend Requirements

- Provide a usable web interface for uploading an image and starting an analysis.
- Display the uploaded image and resulting quality assessment clearly.
- Display the overall quality score and detected issues.
- Show useful information such as severity, confidence, and relevant image statistics.
- Provide a way to view previous analyses or analysis history.
- Handle loading, success, and error states appropriately.
- The interface should be responsive and reasonably polished; visual design is secondary to functionality.

React, Vue, plain HTML/CSS/JavaScript, or another suitable web technology may be used.

7. Expected Analysis Result

The API should return an overall quality assessment and detected issues. The exact response format is left to the applicant.

```
{
"quality_score": 82,
"quality_label": "ACCEPTABLE",
"issues": [{"type": "noise", "severity": "low", "confidence": 0.71}]
}
```

8. Dataset and Training

Applicants may use an appropriate public dataset, a provided dataset, or generate controlled image-quality degradations from clean images. If synthetic degradation is used, describe how training and evaluation data were generated. Evaluation should use unseen images and provide evidence of generalization.

9. Evaluation

Use metrics appropriate to the selected task, such as accuracy, precision, recall, F1-score, ROC-AUC, confusion matrix, anomaly-detection metrics, or regression error metrics. Include failure cases, limitations, and a short discussion of incorrect or uncertain predictions.

10. Explainability

Provide a reasonable explanation for quality decisions. Depending on the chosen model, this may include interpretable image statistics, feature importance, confidence, saliency maps, Grad-CAM, or another suitable technique.

11. Deployment Requirements

- The complete application must be runnable outside the applicant's development environment.

- Provide clear deployment and setup instructions.
- Containerization using Docker is strongly preferred.
- Frontend and backend must communicate correctly in the deployed environment.
- Use environment variables for configurable settings where appropriate.
- Expose a health/status endpoint or equivalent service check.
- Document how the model is loaded and inference is performed after deployment.
- Local Docker Compose deployment is acceptable; cloud deployment is optional. If deployed online, provide the URL.

12. Submission Requirements

- Complete source code for frontend, backend, and AI/ML components.
- README with setup, model/training, API, and deployment instructions.
- Database setup instructions.
- API documentation or example requests.
- Evaluation results and a brief technical explanation.
- Sample images demonstrating different quality conditions.
- Docker/Docker Compose configuration if used.
- Deployed URL, if applicable.

13. Optional / Bonus Work

- Batch image analysis.
- Quality heatmaps or localization of problematic regions.
- Confidence calibration or uncertainty estimation.
- Model versioning.
- Automated backend/frontend tests.
- Performance optimization for simultaneous requests.
- CI/CD workflow.
- Monitoring or logging for the deployed application.

14. Assessment Criteria

Area	Weight
Computer vision understanding and feature reasoning	15%
AI / ML / Deep Learning implementation	25%
Model evaluation and experimental rigor	15%
Backend/API implementation	15%
Frontend functionality and usability	10%
Deployment and reproducibility	10%

Code quality and documentation	10%

15. Note to Applicants

There is no single prescribed implementation. The assessment evaluates technical judgment, computer vision and AI understanding, backend and frontend engineering, deployment skills, and experimental reasoning. Prioritize a robust, well-explained, reproducible solution over unnecessary complexity.